Scrum-projekt med virksomhedscase

Baggrund

I projektet skal I udvikle en applikation med udgangspunkt i en case. Virksomheden præsenterer casen ved projektets begyndelse og deltager igen ved projektets afsluttende demonstration og feedback.

Under projektet fungerer underviseren (Tine) som Product Owner (PO) og kan løbende hjælpe med at afklare produktkrav, prioriteter og afgrænsning.

I arbejder i teams og organiserer udviklingen med Scrum. Formålet er både at udvikle en relevant og fungerende løsning og at demonstrere, at I kan gennemføre en professionel, iterativ systemudviklingsproces.

Teknologisk udgangspunkt

Løsningen skal som udgangspunkt baseres på teknologier, som I allerede har arbejdet med:

- React-frontend
- Java/Javalin-backend
- REST API mellem frontend og backend
- PostgreSQL-database
- Docker og Docker Compose
- GitHub til versionsstyring og samarbejde
- GitHub Projects til planlægning og styring
- GitHub Actions til automatiske tests og checks

Andre teknologier kan anvendes, hvis valget er relevant for casen og aftales med PO.

Roller

Rolle	Varetages af	Ansvar
Stakeholder/kunde	Virksomheden	Præsenterer case, behov og kontekst samt giver afsluttende feedback
Product Owner	Underviseren	Afklarer produktkrav, prioriterer Product Backlog og træffer løbende produktbeslutninger
Udviklere	Hele gruppen	Planlægger, udvikler, tester, integrerer og dokumenterer løsningen
Scrum Master	En studerende	Understøtter Scrum-processen og hjælper teamet med at håndtere hindringer
Tech Lead	En studerende	Facilitere tekniske og artitektoniske drøftelser

Scrum Master er ikke projektleder og skal ikke fordele arbejdet eller bestemme over de øvrige medlemmer. Teamet har i fællesskab ansvar for Sprint Goal og det færdige produkt.

Tech Lead er ikke en Scrum rolle, men en intern ansvarsrolle blandt udviklerne. Rollen kan hjælpe med teknisk koordinering. Tech Lead har ikke beslutningsret over de øvrige udviklkere. Tekniske og arkitektoniske beslutninger samt ansvaret for kodebasen tilhører hele teamet.

Teamet kan vælge at lade rollerne som Scrum Master og Tech Lead gå på skift, men ansvaret skal altid være tydeligt placeret.

Projektets overordnede forløb

1. Kickoff med virksomheden

Virksomhedens præsentation er gruppens vigtigste mulighed for at forstå domænet og det oprindelige behov. I skal derfor aktivt undersøge:

- hvilket problem virksomheden ønsker løst
- hvem brugerne og øvrige interessenter er
- hvordan relevante arbejdsprocesser foregår i dag
- hvilke behov, begrænsninger og kvalitetskrav der findes
- hvad der skaber størst værdi for virksomheden
- hvilke antagelser og uklarheder der kræver yderligere afklaring

Virksomheden afleverer ikke en kravspecifikation. I skal analysere casen og omsætte den til et realistisk produktforslag.

2. Projektopstart

Efter kickoff skal I etablere et første grundlag for udviklingen:

- produktets formål, målgruppe og afgrænsning
- centrale funktionelle og non-funktionelle krav
- en første Product Backlog med prioriterede user stories
- acceptkriterier for user stories
- estimering med story points
- foreløbig datamodel og systemarkitektur
- GitHub-repository, GitHub Project og GitHub Actions
- en fælles Definition of Done
- en kort fælles kodestandard og samarbejdsaftale

Backlog, datamodel og arkitektur er foreløbige og skal kunne udvikle sig, efterhånden som I lærer mere.

3. Iterativ udvikling i sprint

Projektet gennemføres i en række sprints, som skal resultere i et integreret og potentielt anvendeligt **Increment**. Dvs. udvikle vertikale slices, hvor funktionalitet går gennem frontend, REST API, backend og database.

I kan se mødeplan for de konkrete uger i på Moodle.

Uge	Sprint	Indhold
37	Opstart sprint 1	Projektetablering og første user stories
38	Sprint Review 1 + Planning Sprint 2	Demo og planlægning af næste sprint
39	Sprint Review 2 + Planning Sprint 3	Demo og planlægning af næste sprint
40 **28/9**	Sprint Review 3 + Planning Sprint 4	Kort demo og alm. undervisning. Som udgangspunkt ingen nye user stories i sidste sprint, men fokus på kvalitetssikring og automatisering.
41	Sprint Review 4	Sidste sprint reviews. Dette gør vi I fællesskab som videndeling for hele holdet.

4. Afslutning

Projektet afsluttes med en demonstration for virksomheden og underviseren i uge 44 (d. 27/10). Her skal I:

- demonstrere jeres løsning
- forklare væsentlige afgrænsninger og valg
- Kort fortælle hvordan I brugte Scrum, herunder
  - hvad gik godt
  - hvad kunne blive bedre

Scrum og projektstyring

Product Backlog

Product Backlog skal være det samlede, prioriterede overblik over ønsket arbejde. Backlog-items skal være forståelige og tilstrækkeligt afgrænsede til, at de kan anvendes i planlægningen.

User stories skal som udgangspunkt:

- beskrive værdi for en identificeret bruger eller interessent
- have konkrete acceptkriterier
- være prioriterede
- være estimerede, inden de tages ind i et sprint
- opdeles, hvis de er for store eller uklare

Product Backlog er dynamisk. Ny viden, tekniske erfaringer og feedback fra PO kan føre til ændringer i indhold og prioritering.

Refinement og kontakt med PO

I skal løbende forberede kommende backlog-items gennem refinement. Her afklares blandt andet indhold, acceptkriterier, størrelse, afhængigheder og prioritet.

PO træffer beslutninger om **hvad** produktet skal kunne, **hvorfor** det er vigtigt, og hvad der har højest prioritet. Teamet træffer de tekniske beslutninger om **hvordan** løsningen udvikles.

En story bør normalt være forstået, have acceptkriterier og være lille nok, før den vælges til et sprint.

Sprint Planning

Ved Sprint Planning skal hver gruppe (med input fra PO):

- formulere et tydeligt Sprint Goal
- vælge relevante og prioriterede stories fra Product Backlog
- vurdere, hvor meget teamet realistisk kan gennemføre
- nedbryde stories i nødvendige tekniske tasks
- identificere væsentlige risici og afhængigheder

Sprint Backlog skal samlet understøtte Sprint Goal.

Daily Scrum

Daily Scrum afholdes kort og regelmæssigt og bruges til at planlægge og koordinere det næste døgns arbejde mod Sprint Goal.

Mødet skal især afdække:

- om teamet er på vej mod Sprint Goal
- hvad der kræver koordinering
- om der er hindringer eller afhængigheder
- om arbejdsplanen bør tilpasses

Daily Scrum er ikke lange statusrapporter til Scrum Master. Der kræves ikke detaljerede mødereferater, men det vil være hensigtsmæssigt for alle at tage noter fra mødet til brug i jeres individuelle refleksionsrapporter.

Sprint Review

Ved Sprint Review demonstrerer gruppen det fungerende Increment for PO. Reviewet skal bruges til at inspicere produktet, drøfte læring og justere Product Backlog.

Ufærdig funktionalitet bør ikke præsenteres som Done. Feedback og beslutninger fra reviewet dokumenteres kort, især når de fører til ændringer i Backlog eller produktretning.

Sprint Retrospective

Retrospective handler om teamets samarbejde og udviklingsproces. Teamet skal identificere mindst én konkret og realistisk forbedring til næste sprint.

Eksempel: *Vores Pull Requests blev for store og langsomme at reviewe. I næste sprint vil vi integrere mindre ændringer hyppigere*.

Generelle konklusioner som "kommunikationen skal være bedre" er ikke tilstrækkelige uden en konkret handling.

GitHub og samarbejdsworkflow

GitHub skal afspejle det faktiske arbejde og fungere som gruppens centrale samarbejdsplatform.

Gruppen skal beslutte sig for et workflow. Det kunne f.eks. være:

> *Issue → branch → implementering og tests → Pull Request → code review → merge*

Der forventes:

- et fælles repository
- Issues, der beskriver planlagt arbejde
- Meningsfulde commits
- GitHub Project som aktivt planlægnings- og styringsværktøj
- GitHub Actions til automatiske tests og checks

Et muligt board-flow er:

Product Backlog → Ready → In Progress → In Review → Done

Boardet skal opdateres løbende og vise projektets reelle status.

Engineering practices og kodekvalitet

Scrum beskriver ikke i sig selv, hvordan god software konstrueres. I skal derfor vælge og anvende relevante engineering practices.

Som minimum forventes I at forholde jer til:

- Continuous Integration med hyppig integration
- automatiserede tests
- code review
- løbende refactoring
- fælles ejerskab af kodebasen
- en fælles kodestandard
- håndtering af teknisk gæld

Kodestandard

I skal formulere en kodestandard i `CONTRIBUTING.md`. Den bør som minimum forholde sig til:

- navngivning og formattering
- package- og mappestruktur
- endpoints, DTO'er, validering og fejlrespons
- Java exception handling
- branch-navne, commits og Pull Requests

Definition of Done

I udarbejder en fælles Definition of Done. Den bør som minimum sikre, at:

- user storyens acceptkriterier er opfyldt
- funktionaliteten er integreret og kan demonstreres
- koden følger gruppens kodestandard
- relevante automatiske tests består
- relevant dokumentation er opdateret

Arbejde, der ikke opfylder Definition of Done, tæller ikke som en del af sprintets færdige Increment.

Dokumentation

Dokumentationen skal være nyttig for *udvikling*, *overdragelse* og *faglig begrundelse*.

Følgende skal fremgå af repository:

- `README.md` med formål, løsningens hovedfunktioner og opstartsvejledning
- Product Backlog og Sprint Backlogs i GitHub Projects
- user stories og acceptkriterier i Issues
- arkitekturdiagram og begrundelse for arkitekturvalget
- datamodel
- API-dokumentation
- vejledning til at afvikle tests
- kort kodestandard eller `CONTRIBUTING.md`
- få, relevante ADR'er for væsentlige beslutninger
- kort dokumentation af Sprint Goals, Reviews og Retrospectives
- væsentlig feedback fra PO samt ændringer, den førte til

Dokumentationen skal vedligeholdes gennem projektet og stemme overens med den implementerede løsning.

Faglige forventninger

Det forventes, at I kan demonstrere og forklare:

- hvordan virksomhedens problem er omsat til et produkt og en prioriteret backlog
- hvordan Scrum er anvendt til løbende planlægning, inspektion og tilpasning
- hvordan teamet har organiseret samarbejde og versionsstyring
- hvordan løsningen er opdelt i vertikale, integrerede increments
- hvordan arkitektur- og designvalg understøtter kravene
- hvordan automatiserede tests, CI og code review bidrager til kvalitet
- hvordan feedback og ny viden har ændret produktet eller processen
- hvordan løsningen kan startes, testes, fejlfindes og videreudvikles

De anvendte teknologier og udviklingsværktøjer er ikke mål i sig selv. Det afgørende er, at I kan begrunde sine valg og demonstrere, hvordan de har understøttet projektstyring, samarbejde, kvalitet og udviklingen af løsningen.

Aflevering

Den endelige *fælles aflevering* af projektet sker på Wiseflow (deadline i december), og skal indeholde links til Github repo + GitHub Project. Husk at gøre dem synlige (public visibility).

Men I første omgang skal I aflevere links på Moodle: Scrum projekt links. Gør det allerede nu, når repo og projektboard er oprettet!

I den endelige aflevering på Wiseflow bliver der også en *individuel aflevering* på 4 – max 5 normalsider, der skal indeholde jeres individuelle refleksion over projektet (samt det næste Kanban projekt).

Se tekst om refleksion i materialemappen.